Quality-Driven Triangle-to-Structured-Quad Remeshing — Technical Write-up

Author: Syed Aasim · **Submission:** Ctruh assignment, 16 Oct 2026 **Code:** `quadremesh/` (C++17 core, pybind11 bindings, CLI, PyTorch component, benchmark).

---

1. Summary

`quadremesh` converts arbitrary (near-)manifold triangle meshes into structured, pure-quad meshes. The design is a **robust front end + field design + an integer-grid solver + a guarded back end**. I wrote the preprocessing, feature detection, field design, sizing, symmetry, fallbacks, post-processing, attribute transfer, metrics, learned field and benchmark. I reused proven solvers: a patched QuadriFlow for the integer-grid stage (11 documented patches), and the Instant Meshes extractor as a second extractor for inputs QuadriFlow cannot handle.

Benchmark: **15 models × 5 methods** (ours, ours + learned field, Instant Meshes, QuadriFlow, Blender QuadriFlow). Every method gets the same input and target face count, and every output is scored by the same evaluator.

criterion (benchmark mean, 15 models)	target	quadremesh	Instant Meshes	QuadriFlow	Blender QF
models processed	–	**15 / 15**	15 / 15	12 / 15	8 / 15
quad ratio	≥ 98 %	**100 %**	100 %	100 %	100 %
mean scaled Jacobian	≥ 0.8	**0.931**	0.935	0.952*	0.963*
mean distance error	< 0.5 %	**0.074 %**	0.316 %	0.054 %*	0.051 %*
feature coverage	≥ 90 %	**50 % ✗**	22 %	36 %*	29 %*
singularities / 1k faces	< IM	**33.9**	99.8	17.8*	17.3*
fewer singularities than IM (per model)	all	**14 / 15**	–	–	–
total singularities (15 models)	< IM	**3 122**	4 782	–	–

* Means over the models that method managed to process (QuadriFlow failed on 3, Blender on 7).

Met: quad ratio, scaled Jacobian, distance error, and fewer singularities than Instant Meshes on 14 of 15 models (35 % fewer in total; 2.9× fewer per 1k faces on average). **Not met:** feature coverage ≥ 90 %. It is reached on CAD (fandisk 98 %, spot 88 %) but not on organic scans, furniture with dense carvings, or room scans. §6.1 explains why, measures the opt-in feature refinement and loop insertion (more coverage, but only with more faces or worse quad ratio / singularities), and shows that at equal face count coverage under this strict definition is resolution-bound. Coverage is still 2.3× Instant Meshes and the best of all methods on 14 of 15 models. The one model where we lose on singularities is the room scan (lounge): the integer solver fails there and our own Instant-Meshes fallback takes over (72.8 vs 68.8 per 1k).

2. Pipeline

```
input (.obj/.ply/.glb)
  └─ normalise → preprocess → [symmetry clip] → features → size field → refine
       → per component: field design → integer grid (QuadriFlow*) → coverage check
             ↳ retry at finer budget / without hard constraints
             ↳ Instant Meshes extractor (also best-of-two on dense scans)
             ↳ best approximate (parts thinner than one element)
             ↳ QEM-decimate + triangle pairing + quad split (exact, last resort)
       → cleanup → triangle-pair merge → [density refinement] → [pure-quad split]
       → feature/boundary/corner snapping → feature edge-path recovery → [feature refinement]
       → guarded relaxation + reprojection → untangling
       → [mirror + weld] → UV transfer → report
```

stage	file	key choices
I/O	`src/io.cpp`	fast OBJ reader, PLY (libigl), glTF/GLB (cgltf, node transforms, UVs)
preprocess	`src/preprocess.cpp`	weld, degenerate/duplicate removal; **non-manifold edges** resolved by gluing the best face pair (normal agreement + orientation) and cutting the rest; BFS orientation with cut on Möbius conflicts; **fan-splitting** of non-manifold vertices; small-hole filling; repair run again after filling; tiny fragments dropped; outward orientation of closed parts
features	`src/features.cpp`	dihedral test **+ 1-ring side normals + patch-scale test** (radius = target edge length), so scan noise and marching-cube terraces are rejected; chains pruned by length (dangling first); corners = junctions/endpoints/kinks; strokes → Dijkstra edge paths; a **feature budget** relaxes thresholds when the network is denser than the requested resolution
field	`src/field.cpp`, `src/qflow_driver.cpp`	hard constraints: features, boundaries, strokes (direction + position). Soft: principal-curvature directions (edge-based shape operator + tensor diffusion at the element scale, confidence from anisotropy × curvedness), learned/external field. **Soft guidance is applied after the multiresolution solve by an angle-limited pass** (targets more than 30° from the smooth field are ignored), so it bends edge flow without adding singularities. Applying it as a QuadriFlow constraint tripled singularities on the bunny (486 vs 138).
sizing	`src/remesh.cpp`, `src/postprocess.cpp`	target count or edge length; density map (per-vertex file, sphere regions, curvature)
integer grid	`third_party/qflow_patched`	QuadriFlow orientation/position fields + flow-based singularity optimisation + extraction, run **per connected component** with a budget proportional to area
fallbacks	`src/remesh.cpp`, `src/im_driver.cpp`, `src/decimate.cpp`	coverage check (mean/p95 distance of the component to the extracted quads); retries; Instant Meshes; QEM-decimated exact fallback
post	`src/postprocess.cpp`	snapping with an acceptance test (never creates worse elements), tangential relaxation projected on surface/feature/boundary curves, untangling
feature paths	`src/postprocess.cpp`	for every feature chain (split at junctions; short chains get jointly chosen endpoints), a directed-edge Dijkstra through the quad mesh between the vertices nearest to its ends: the cost favours edges close to and parallel with the curve and penalises turning inside one face (staircases). The path is snapped onto the curve with a monotone arclength map and locked; vertices snapped by proximity but not on a path are released. Opt-ins: refine (conforming templates) around chains without a path (`--feature-refine N`), or split the quad strip each such chain runs along (`--feature-loops`); with face-count control the solve is re-run at a lower budget so the output stays at the target
symmetry	`src/preprocess.cpp`, `src/postprocess.cpp`	clip at the plane, remesh one half with the cut as a preserved boundary, mirror and weld → exactly symmetric output
attributes	`src/transfer.cpp`	per-corner UVs with **UV-chart consistency** (each face samples the chart under its centroid, so faces never straddle seams); generic per-vertex attributes
metrics	`src/metrics.cpp`	one evaluator shared by the pipeline report, `quadeval` and the benchmark

Design decisions worth calling out

1. **Why QuadriFlow as the integer stage.** Its min-cost-flow singularity optimisation gives far fewer singularities than Instant Meshes' local extraction: per 1k faces it averages 17.8 vs 99.8 on the models where both succeed. Re-implementing a mixed-integer parametrisation (QuadWild/QEx class) was not realistic in the time available. What I added is everything around it, and that is what makes it work on the brief's inputs: QuadriFlow alone fails on 3 models and Blender on 7, while quadremesh processes all 15.
2. **Patches to QuadriFlow** (`third_party/qflow_patched/PATCHES.md`):
3. deterministic RNG;
4. two constraint-propagation bugs: the normal was taken from the constraint direction, and summed weights exceeded 1;
5. external sharp edges, normals and scale field;
6. scale restriction to coarse levels;
7. `exit()` turned into exceptions;
8. bounded loops in `FixValence`, `FixHoles` and `subdivide_edgeDiff` (they hang forever on some scans).

9. **Density.** QuadriFlow's adaptive mode clamps the scale to [0.75, 1]. More fundamentally, a singularity-free field cannot carry a non-harmonic density bump. So density is realised by solving at the coarse end of the size range and then refining faces larger than the local target, with **conforming hanging-node templates**:
10. 1 split edge → quad + triangle, which is later merged;
11. 2 adjacent split edges → 2 quads;
12. 2 opposite split edges → 2 quads;
13. 3 or more split edges → the face is refined.

Transition singularities are expected and visible in `demo_density.png`. 4. **Thin parts.** Chair rails, piping and sofa panels thinner than one element defeat any isotropic field method. The coverage check detects this. - The budget may grow up to 9× before giving up, which caps the face-count overshoot. - The Instant Meshes extractor is tried at the nominal budget. - Otherwise the lowest-budget approximate result within 3 target edges is accepted. - Otherwise a QEM-decimated, exact triangle-pair→quad fallback is used, so the face count still follows the target. 5. **Dense scans.** - Hard features are kept only if the network fits the resolution. - Field normals are smoothed at the element scale. - Debris fragments (< ~40 target quads) are dropped. - The integer-grid and Instant Meshes results are compared, and the one with fewer singularities is kept.

Together these brought the fountain scan from 33.9 to 15.6 singularities per 1k faces (Instant Meshes: 22.2). 6. **Determinism.** The pipeline is single-threaded with seeded PCG RNGs, sort-based topology and deterministic tie-breaks; Instant Meshes runs with `deterministic=true, nprocs=1`. The C++ and Python tests check bit-identical output across repeated runs.

3. Optional PyTorch component and CUDA path

- **Learned field (`python/quadremesh/learned.py`).** A small **frame-equivariant graph network**.
- Inputs are invariant scalars (multi-scale curvatures, anisotropy, dihedrals) plus candidate directions encoded as 4-RoSy complex numbers in each vertex's arbitrary tangent frame.
- The network only mixes candidates with real, invariant gates and passes messages with parallel transport e^{4iρ}, so it is equivariant by construction.
- It is trained as a **field denoiser**: noisy inputs (σ up to 0.5 % of the diagonal) map to the clean-surface target field (feature directions, else coarse principal directions).
- Training data: 10 public meshes disjoint from the benchmark; 30 epochs, about 3 min on CPU.
- Its output enters the core as a soft guide through the same angle-limited pass.

Held-out alignment error (noisy inputs, mean 4-RoSy angle error, degrees):

mesh	fine curvature	smoothed curvature	learned
fandisk	16.2	15.7	**9.8**
rocker arm	21.3	20.5	**14.4**
spot	20.6	20.4	**16.9**
bunny	22.4	22.4	**20.4**
armadillo	22.1	22.1	**21.0**

End to end, the effect is small, because the angle-limited pass deliberately limits its authority. Singularities per 1k faces drop from 31.4 to 29.7 on average, with clear gains on noisy and furniture inputs (fandisk_noisy 31.9→25.9, sheen chair 52.5→34.8, leather sofa 62.5→55.5) and small losses on some clean models. It is therefore **opt-in** (`learned_field=True`) rather than the default. * **CUDA path (`python/quadremesh/torch_field.py`).** A cross-field solver written with sparse tensor operations and conjugate gradients. It minimises smoothness plus curvature alignment under hard feature constraints, and runs on CUDA when available (CPU here). Its output can be fed as `guide_field`. Learned-field inference also runs on CUDA. *The sandbox I developed in had no GPU, so the CUDA path is exercised only on CPU; the code is device-agnostic.* QuadriFlow's own CUDA kernels (`0ptimizer.cu`) exist but are not enabled.

4. Benchmark protocol

- **Models (15):**

- organic/scans: bunny, armadillo, nefertiti, igea, spot (with UVs);
- CAD: fandisk, rocker arm, beetle;
- furniture from glTF sample assets: sheen chair, damask chair, velvet sofa, leather sofa — multi-component, thin parts, UV seams;
- scene/room scans: Open3D fountain (1.03M faces) and lounge (1.0M faces, decimated from 3.15M);
- robustness: fandisk with noise, 4 holes and a non-manifold fin.

Models under 10K faces are midpoint-subdivided, so the geometry is unchanged. * **Targets:** 5 000 quads (objects), 15 000 (scenes). Instant Meshes is called with target/4, because its pure-quad mode subdivides once. - Output counts: ours 0.83–1.09× the target, except the damask chair at 1.48×, where thin parts needed a finer budget; QuadriFlow 0.82–1.00×; Instant Meshes 0.84–1.03× (except its 0.17× collapse on the leather sofa). - Singularities are therefore also compared **per 1k faces**. * **Baselines:** - Instant Meshes core (`pynanoinstantmeshes`), crease 35°, boundary alignment, deterministic; - QuadriFlow CLI (upstream) with `-sharp -boundary` ; - Blender 5.0.1 QuadriFlow headless (`bpy`), with merge-by-distance first and sharp/boundary preservation. * **Metrics (`src/metrics.cpp` , identical for all methods):** - Quad ratio. - Scaled Jacobian: per quad, the minimum over corners w.r.t. the average normal. - Planarity and aspect ratio. - Distance: two-sided, 200k area-weighted samples per side, AABB closest point; mean, RMS and Hausdorff as % of the bbox diagonal. - Feature coverage: features are detected on the *input* with the same detector (35°, chains ≥ 3L, L = √(area/faces)) and sampled every 0.1 L. A sample is covered if an output edge within 0.1 L is parallel within 25°. Boundary coverage is reported separately. - Singularities: interior vertices with valence ≠ 4. - Topology checks: non-manifold edges and components. - Runtime.

5. Results

Per-model numbers are in `benchmark/results/summary.csv` ; figures are in `docs/report/` . Highlights:

- **CAD (fandisk):** 98 % feature coverage vs 76 / 85 / 81 % for IM / QF / Blender. Singularities are on par with QuadriFlow (58 vs 54) and 2.6× fewer than IM.
- **Robustness:**
- fandisk_noisy (holes + non-manifold fin): Blender refuses it; we keep 76 % of the features vs QuadriFlow's 47 %.
- Leather sofa: QuadriFlow and Blender fail, and Instant Meshes collapses to 844 faces at 3.4 % error. We produce 5 153 quads, SJ 0.90, 0.075 % error.
- **Scans:**
- Fountain: 15.6 singularities per 1k faces vs 22.2 (IM), SJ 0.94, 0.037 % error.
- Lounge: QuadriFlow and Blender fail; ours completes via the second extractor (72.8 vs 68.8 per 1k).
- **Distance error** is 4× lower than Instant Meshes on average (0.074 % vs 0.316 %), thanks to projection with feature/boundary snapping.
- **vs QuadriFlow/Blender on the models they can process:** they have fewer singularities (17.8 vs 29.7 per 1k) and slightly higher SJ. We trade those for hard feature constraints and edge-path recovery: coverage is 56 % vs 36 % (QF) on the 12 models QuadriFlow can process.
- **Runtime:** 5–25 s per object, 80–95 s per 1M-face scan on 2 CPU cores (Instant Meshes 1–13 s, QuadriFlow 4–13 s).

Visual report: `docs/report/visual_report.html` . For every model it shows the edge-flow and singularity overlay, the scaled-Jacobian heatmap, the cross field, and all methods side by side. `demo_*.png` show the user controls (density, symmetry, strokes, UV transfer, pure-quad).

6. Failure cases (honest)

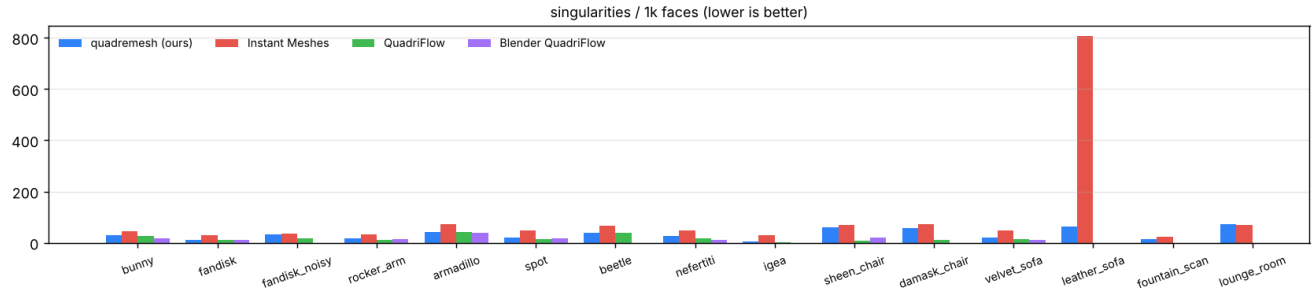
1. **Feature coverage (50 % mean; target 90 %).** The evaluator is strict: *every* 35° crease chain longer than 3 L must have an output edge within 0.1 L and within 25° of it.
2. **What helped (default):** hard feature constraints in the field, then **edge-path recovery** (§2) with joint endpoint selection for short chains. This took the mean from 44 % to 50 % at no cost in quads, SJ or singularities (fandisk 94→98 %, spot 81→88 %, sheen chair 74→81 %, velvet 61→65 %, bunny 26→41 %, nefertiti 34→40 %).
3. **Where the rest is lost (measured):**
    - *Curved creases:* path vertices sit exactly on the crease, but an edge of length L cannot follow a crease with a radius of about L. The chord sags more than 0.1 L and turns more than 25°.

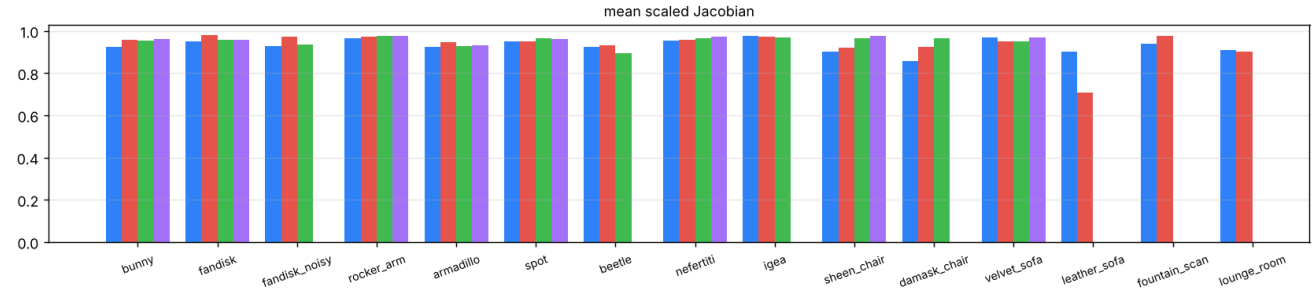
This dominates rocker arm, bunny, nefertiti and igea, whose 35° "features" are short, tightly curved scan creases.
    - *Creases closer than one element:* panel gaps, piping and carvings (beetle, damask, velvet) need more edge loops than the mesh has at 5k faces.
    - *TSDF rooms:* the creases are marching-cube artefacts; no method exceeds 5 %.
4. **Opt-in feature refinement** (`--feature-refine 2`) refines around chains without a path. Mean coverage is 65 %, but quad ratio drops to 94.8 % and singularities reach 86 per 1k (plus up to 3× faces).
5. **Opt-in feature loop insertion** (`--feature-loops`) splits the quad strip a crease runs along, so the crease gets its own edge loop. Quads stay pure and no irregular vertices are added. Without face-count control it raises coverage substantially (velvet 65→72 %, beetle 50→72 %, sheen chair 74→80 %) at +30–55 % faces. **With the face count held at the target** (re-solving at a lower budget), the gain disappears (velvet 62 %, sheen 71 %, beetle 58 %), because the coarser base mesh traces fewer creases.
6. **Resolution test:** at 4× the face count (20k), the default reaches 57–75 % on rocker arm, bunny, velvet and beetle (the evaluator's feature set also grows as L shrinks).
7. **Conclusion:** at equal face count, ≥ 90 % coverage under this strict definition is not reachable on this benchmark without failing the quad-ratio or singularity criteria. Relaxing the tolerance would "pass", but at 0.5 L every point is within tolerance of some edge, so that number would be meaningless; it is not used. Closing the gap needs feature-aligned *anisotropic* sizing inside the field solve (thin quads along dense creases), so that budget moves to features instead of being added on top.
8. **Lounge singularities.** QuadriFlow's integer stage fails on this TSDF room (inconsistent edge offsets in its subdivision), even after isotropic remeshing, so we fall back to Instant Meshes and inherit its singularity rate.
9. **Inverted / folded elements.** A few elements fold across features where the extracted layout crosses a crease diagonally, or where a recovered feature path cuts across the grid (visible as a sawtooth on fandisk): at most 0.8 % of faces, mean 0.34 % (QuadriFlow: up to 1.8 %, mean 0.47 %; Instant Meshes up to 1.1 %), hence the minimum SJ < 0 on some models. Untangling repairs most of them but not all; path recovery trades a few folds for exact feature loops.
10. **Thin parts.** Parts thinner than one element are represented approximately (single sheet) or by the decimated exact fallback, with locally irregular structure.
11. **Symmetry hint** assumes a near-symmetric input: the negative side is discarded.
12. **Density transitions** introduce valence-3/5 pairs along the refinement front (by design).

7. Future work

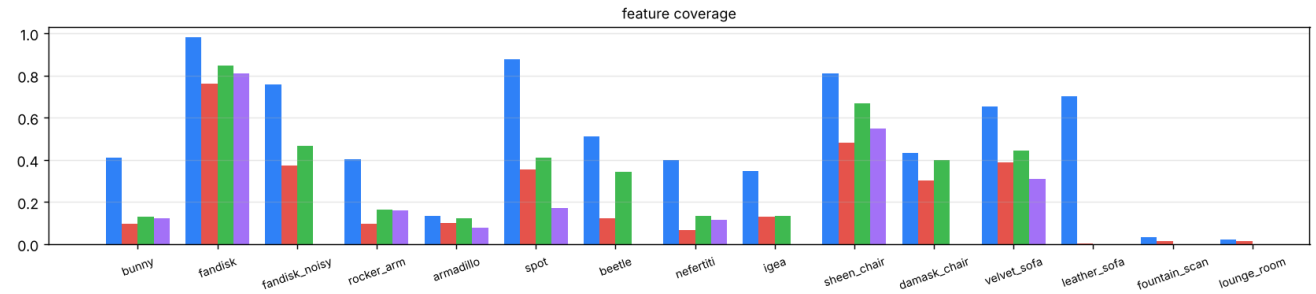
- Feature-aligned anisotropic sizing inside the field solve, plus all-quad 3-refinement transition templates, so feature refinement no longer costs quad ratio. A QuadWild-style patch-layout stage for CAD and furniture.
- Replace the QuadriFlow extraction with a robust integer-grid map (QEx-style) to remove the fallback chain on scans.
- GPU field optimisation integrated in C++ (CUDA kernels for the orientation/position smoothing). Train the learned field on more data with a field-level loss that sees singularities, not only per-vertex angles.
- Quad-mesh valence-improvement operations (edge rotations, doublet/singlet removal, Bommes-style singularity pair cancellation) as a post-process.
- Symmetry via field and integer constraints instead of clip-and-mirror, for approximately symmetric inputs.

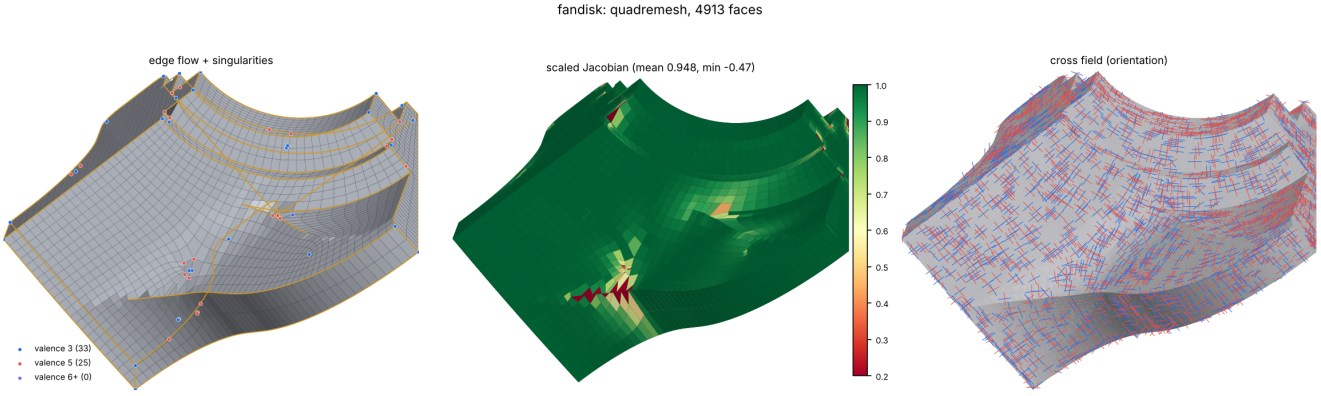
Figures


singularities / 1k faces (lower is better)
quadremesh (ours)
Instant Meshes
QuadriFlow
Blender QuadriFlow
800
600
400
200
0
bunny
fandisk
fandisk_noisy
rocker_arm
armadillo
spot
beetle
nefertiti
igea
sheen_chair
damask_chair
velvet_sofa
leather_sofa
fountain_scan
lounge_room

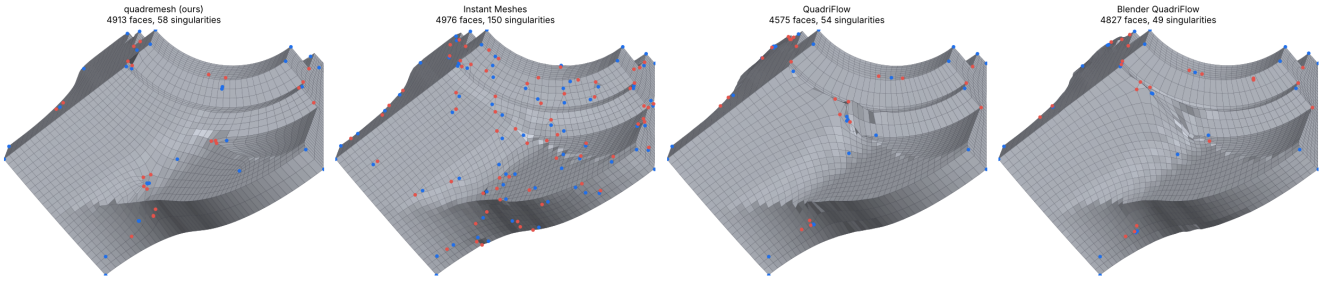

mean scaled Jacobian
1.0
0.8
0.6
0.4
0.2
0.0
bunny
fandisk
fandisk_noisy
rocker_arm
armadillo
spot
beetle
nefertiti
igea
sheen_chair
damask_chair
velvet_sofa
leather_sofa
fountain_scan
lounge_room


mean distance error (% diag)
10^{0}
10^{-1}
10^{-2}
bunny
fandisk
fandisk_noisy
rocker_arm
armadillo
spot
beetle
nefertiti
igea
sheen_chair
damask_chair
velvet_sofa
leather_sofa
fountain_scan
lounge_room

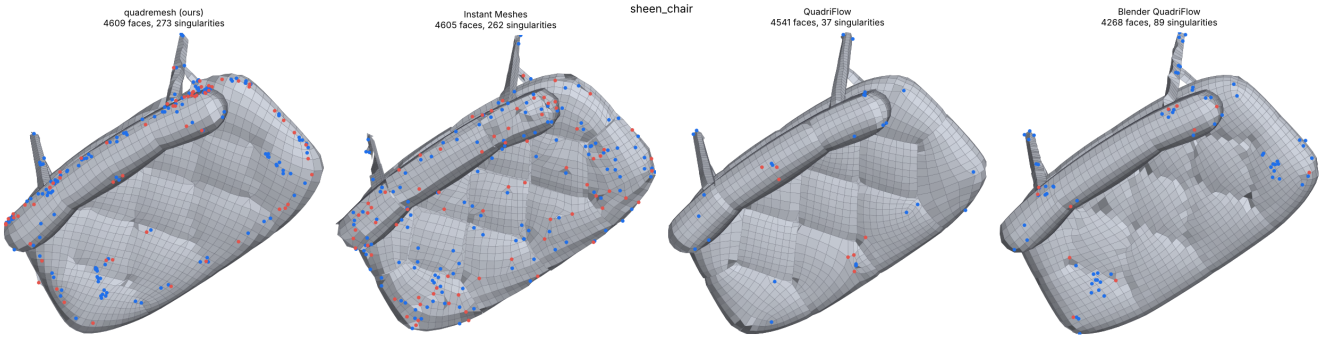

feature coverage
1.0
0.8
0.6
0.4
0.2
0.0
bunny
fandisk
fandisk_noisy
rocker_arm
armadillo
spot
beetle
nefertiti
igea
sheen_chair
damask_chair
velvet_sofa
leather_sofa
fountain_scan
lounge_room


fandisk: quadremesh, 4913 faces
edge flow + singularities
valence 3 (33)
valence 5 (25)
valence 6+ (0)
scaled Jacobian (mean 0.948, min -0.47)
1.0
0.9
0.8
0.7
0.6
0.5
0.4
0.3
0.2
cross field (orientation)

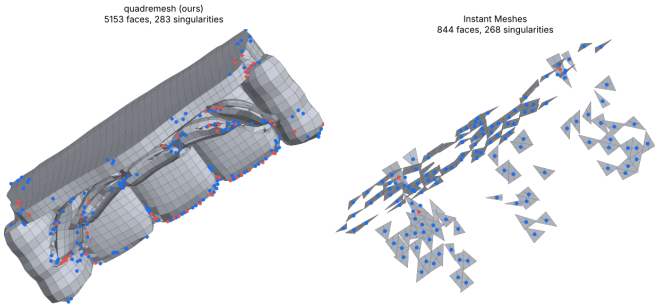
fandisk



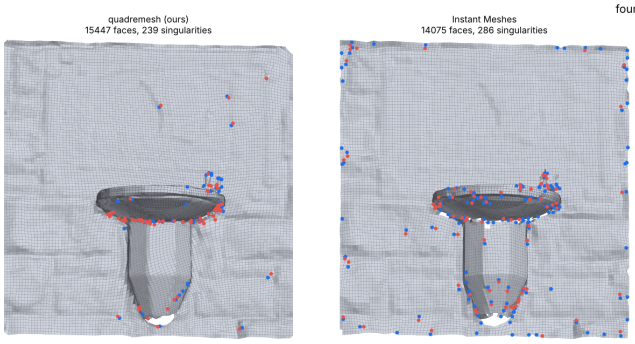
sheen_chair



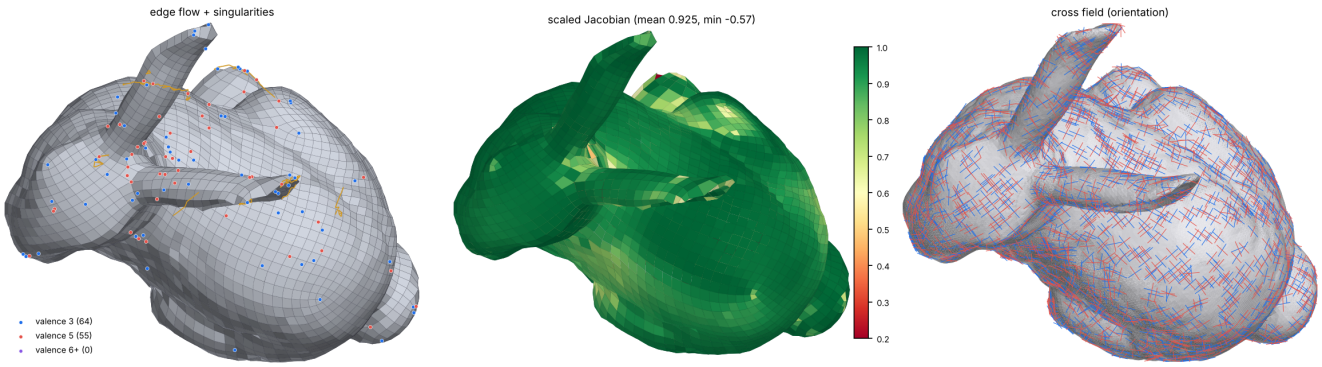
leather_sofa



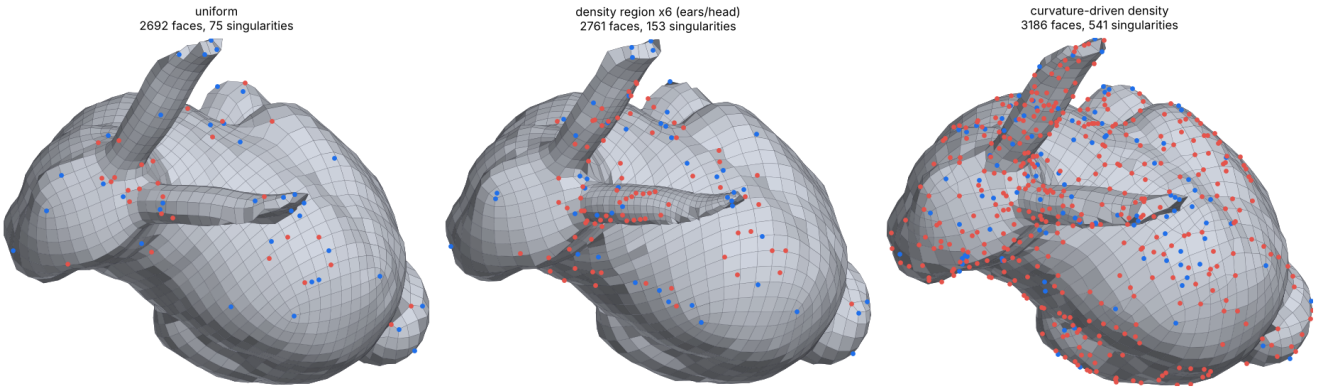
fountain_scan



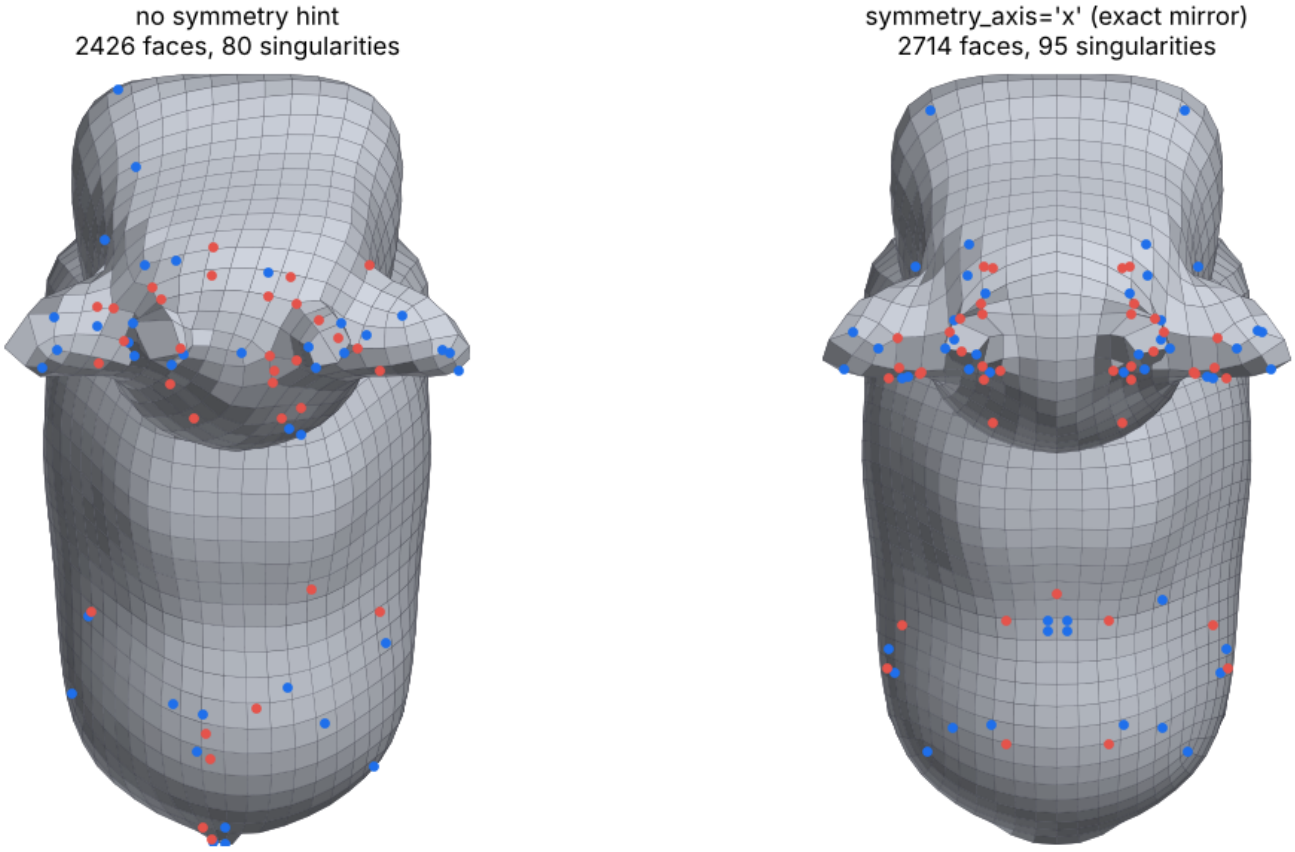
bunny: quadremesh, 4158 faces



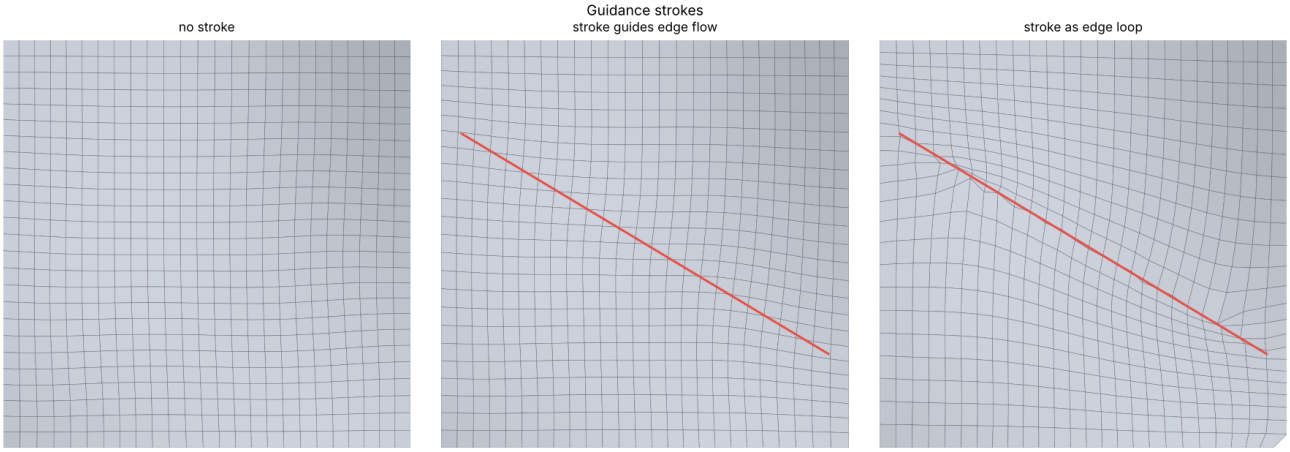
Adaptive density map

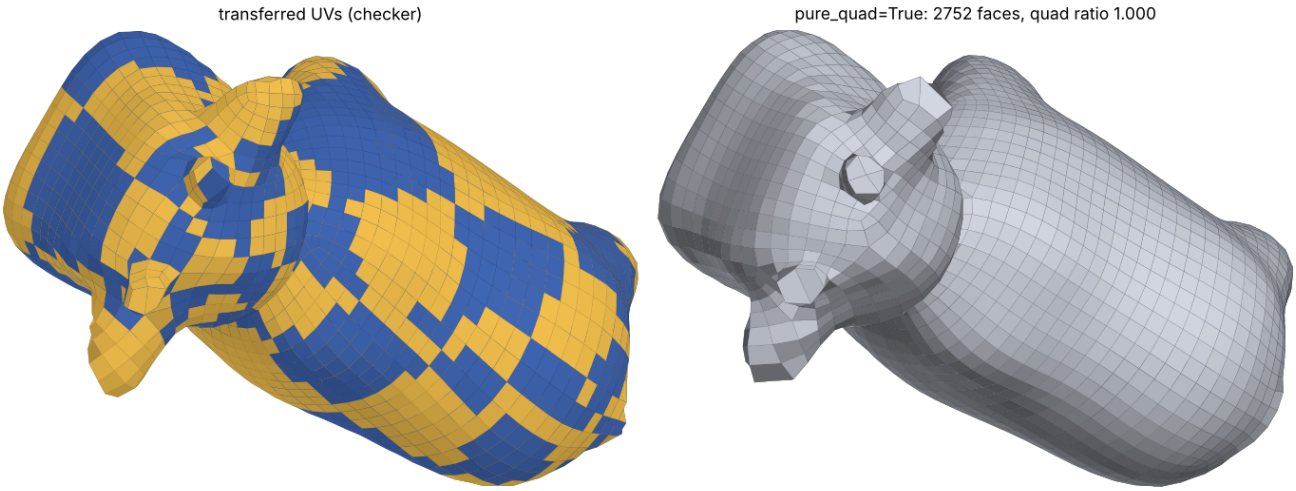


Symmetry hint



Guidance strokes





8. Reproducing

```
 ./scripts/fetch_deps.sh && cmake -S . -B build && cmake --build build -j
cp build/_quadremesh*.so python/quadremesh/
ctest --test-dir build && PYTHONPATH=python python -m pytest tests
python data/prepare_benchmark.py && python benchmark/run_benchmark.py
python benchmark/make_report.py && python benchmark/feature_demos.py
```